Why Hard Negative Mining Strategies Don't Differentiate for Code Retrieval at Scale

Anonymous

May 14, 2026

Abstract

Hard negative mining is widely believed to be critical for training contrastive code retrievers, yet the field lacks controlled comparisons isolating the effect of mining strategy from confounding factors such as training scale, encoder choice, and evaluation protocol. We present a systematic study of four negative mining strategies—random, embedding-distance, dataflow-aware, and BM25—across two decoder-only code encoders (Qwen2.5-Coder-3B and StarCoder2-3B) with 9 random seeds per condition. Our central finding is *negative*: at full training scale ($\sim$44K instances), no mining strategy reliably outperforms random baselines. The largest observed effect is $+0.20$ percentage points (pp) for embedding mining on Qwen ($p = 0.58$), with 95% confidence intervals spanning zero for all comparisons. Strategy rankings are unstable across pool sizes (Kendall $\tau = 0.33$ between pool 50 and 500), and downstream SWE-bench Lite evaluation shows no differentiation ($\Delta < 0.67$pp, bootstrap CI width $\sim$5pp). We provide a mechanistic explanation: fine-tuning acts as a *geometry equalizer*, collapsing pretrained margin differences across strategies by 65–78%. A data-scaling ablation reveals that mining advantages observed at pilot scale ($+1.3$pp at 1K instances) vanish monotonically with training set size, suggesting that scale substitutes for curation in contrastive code retrieval. We additionally show that BM25 negatives perform indistinguishably from embedding negatives ($p > 0.42$), preventing closure of the argument that prior work's reported harm from hard negatives was caused by BM25-proxy methodology. Our results caution against over-investing in mining strategy optimization and redirect attention toward training scale and inference-time structural methods.

1 Introduction

Repository-level code retrieval—selecting the relevant source files from a large codebase given a bug report or issue description—is a critical bottleneck in automated software repair pipelines [1]. Contrastive learning has emerged as the dominant paradigm for training dense code retrievers, where the choice of *hard negatives* is believed to strongly influence retrieval quality.

Prior work offers conflicting guidance on hard negative mining for code. CoCoHaNeRe reports that embedding-distance negatives *harm* code search by up to 27 percentage points in R@1, advocating instead for keyword-based retrieval of negatives. However, this finding was obtained using BM25 as a proxy for embedding distance, raising the question of whether the reported harm reflects true embedding mining or confounds introduced by the proxy. Meanwhile, the contrastive learning literature in NLP and vision consistently finds that harder negatives improve representation quality, at least up to a difficulty threshold beyond which false negatives degrade training.

We set out to resolve this contradiction through a controlled study comparing four negative mining strategies for code retrieval: (1) random sampling, (2) true embedding-distance mining, (3) structure-grounded (dataflow-aware) mining using inter-procedural call-graph analysis, and (4) BM25-based mining. We train with each strategy on two decoder-only code encoders with distinct pretrained representation geometries—Qwen2.5-Coder-3B (saturated, pos_sim $= 0.986$) and StarCoder2-3B (healthy, pos_sim $= 0.798$)—using 9 random seeds per condition to obtain reliable confidence intervals.

Our findings are predominantly negative, which we argue is itself a contribution:

1. **No strategy differentiates at scale.** At full training scale ($\sim$44K instances), all mining strategies produce statistically indistinguishable R@1 on both encoders. The largest effect ($+0.20$pp) has $p = 0.58$.
2. **Representation geometry convergence.** Fine-tuning collapses pretrained margin differences by 65–78%, explaining why strategies that produce different negative difficulties in the pretrained space converge after training.
3. **Scale substitutes for curation.** Mining advantages observed at 1K instances ($+1.3$pp) vanish monotonically as training data grows, reaching $+0.2$pp at 5K instances.
4. **BM25 $\approx$ embedding negatives.** BM25 and embedding-distance negatives are statistically indistinguishable ($p > 0.42$), preventing attribution of prior work's reported harm to the BM25-proxy methodology.
5. **Unstable strategy rankings.** Rankings flip across pool sizes (stability $= 25\%$), temperatures, and encoders, indicating that reported differences in prior work may be evaluation artifacts.

These results caution practitioners against investing GPU time in sophisticated mining pipelines when simple random sampling suffices at scale, and redirect attention toward training data volume and inference-time structural navigation methods.

2 Related Work

Hard Negative Mining for Retrieval. Hard negative mining has a long history in metric learning and dense retrieval. ANCE [2] periodically refreshes negatives using the current encoder, DPR [3] uses BM25 negatives for open-domain QA, and curriculum-based approaches gradually increase negative difficulty during training. For code, CoCoHaNeRe reports that embedding-distance negatives harm code search, but uses BM25 as a proxy rather than true embedding distance. Our work directly tests this claim with proper controls.

Code Retrieval. Dense retrieval for code has been explored with encoder-only models (GraphCodeBERT, CodeBERT) and more recently with decoder-only models fine-tuned for retrieval (SweRank, SWE-Fixer). These systems typically use random or in-batch negatives without systematic comparison of mining strategies. RepoBench [4] provides a cross-file retrieval benchmark that we use for external validation.

Structure-Aware Code Understanding. Graph-based approaches to code operate at either training time or inference time. Training-time methods (GraphCodeBERT) encode structural features into the model. Inference-time methods (CodexGraph, LocAgent, RANGER) navigate code structure during retrieval using the query context. Our dataflow-aware mining falls in the training-time category; our null result suggests inference-time approaches may be more effective for leveraging structure.

3 Method

3.1 Problem Setting

Given a repository $\mathcal{R}$ with source files $\{f_1, \ldots, f_n\}$ and a query q (e.g., a bug report), the retrieval task is to rank files by relevance. We train a contrastive encoder E_θ using InfoNCE loss with temperature τ:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, f^+)/\tau)}{\exp(\text{sim}(q, f^+)/\tau) + \sum_{i=1}^{K} \exp(\text{sim}(q, f_i^-)/\tau)} \tag{1}$$

where f^+ is the gold file and $\{f_i^-\}_{i=1}^K$ are negatives selected by the mining strategy.

3.2 Negative Mining Strategies

We compare four strategies that differ only in how $\{f_i^-\}$ are selected:

Random. Sample K negatives uniformly from non-gold files in the repository.

Embedding-distance. Using the pretrained encoder, compute cosine similarity between the query and all non-gold files. Select the top-K most similar as hard negatives.

Dataflow-aware. Using inter-procedural call-graph analysis via CodeQL, identify files that are *not* reachable from the gold file within 2 hops. Among these structurally disconnected files, select the top-K by embedding similarity. This produces negatives that are semantically similar but structurally unrelated—the intuition being that structural disconnection is a stronger signal of irrelevance than embedding distance alone. Structural data is available for 34% of training instances; the remaining 66% fall back to embedding-distance mining.

BM25. Tokenize query and files, compute BM25 scores, select top-K non-gold files. This replicates the methodology attributed to "embedding negatives" in prior work.

3.3 Encoders

We fine-tune two 3B-parameter decoder-only code models using LoRA:

- **Qwen2.5-Coder-3B**: Pretrained with Fill-in-the-Middle. Exhibits saturated representation geometry (pos_sim $= 0.986$, margin $= 0.074$ for random negatives).
- **StarCoder2-3B**: Pretrained with Fill-in-the-Middle on a different corpus. Healthier geometry (pos_sim $= 0.798$, margin $= 0.194$ for random negatives).

3.4 Training Protocol

All models are trained on $\sim$44K instances from SWE-smith with $K = 3$ negatives, $\tau = 0.05$, LoRA rank 16, for 3 epochs. Each strategy $\times$ encoder combination is trained with 9 random seeds. Evaluation uses R@1 at pool size 50 on a held-out validation set unless otherwise noted.

4 Experiments

4.1 Main Result: No Strategy Differentiates at Full Scale

Table 1 presents the central result. On Qwen, embedding mining achieves $+0.20$pp over random ($p = 0.58$); on StarCoder, it achieves -0.35pp ($p = 0.47$). Neither difference is statistically significant, and the signs are inconsistent across encoders—a signature of noise, not signal. Dataflow-aware mining shows slight harm on both encoders (-0.45pp Qwen, -1.50pp StarCoder), with the StarCoder result reaching $p = 0.015$ before multiple-testing correction but not surviving Bonferroni adjustment across the study's comparisons.

Seed-to-seed variance ($\sigma \approx 0.009$–0.014) exceeds the strategy effects in all cases. The confidence intervals for all pairwise strategy comparisons include zero: embedding $-$ random on Qwen: $[-0.62, +1.02]$pp; embedding $-$ random on StarCoder: $[-1.4, +0.7]$pp; dataflow $-$ random on StarCoder: $[-3.1, +0.1]$pp.

Table 1: Full-scale R@1 by strategy and encoder (9 seeds, pool=50). No strategy significantly outperforms random.

Encoder	Strategy	Mean R@1	Std	95% CI	Δ vs Rand
Qwen	Random	0.529	0.009	[.522, .536]	—
	Embedding	0.531	0.009	[.524, .538]	+0.20pp
	Dataflow	0.525	0.011	[.516, .533]	−0.45pp
StarCoder	Random	0.443	0.011	[.435, .451]	—
	Embedding	0.440	0.012	[.430, .449]	−0.35pp
	Dataflow	0.428	0.014	[.418, .438]	−1.50pp

Table 2: Four-strategy comparison including BM25 baseline. BM25 and embedding negatives are indistinguishable.

Encoder	Random	Embedding	Dataflow	BM25	BM25−Emb
Qwen	0.529	0.531	0.525	0.528	−0.35pp
StarCoder	0.443	0.440	0.428	0.437	−0.25pp

4.2 BM25 Negatives Match Embedding Negatives

Table 2 adds BM25 as a fourth strategy. BM25 negatives are statistically indistinguishable from embedding negatives on both encoders (Qwen: -0.35pp, $p = 0.42$; StarCoder: -0.25pp, $p = 0.55$). This prevents closing the argument that prior work's reported harm from hard negatives was caused by the BM25-proxy methodology specifically. The high overlap ($\sim$60%) between BM25 and embedding top-K negatives at full scale explains this convergence.

We observe that CoCoHaNeRe's reported -27pp harm was not reproduced in our controlled setup with either BM25 or embedding negatives, suggesting the effect is attributable to other experimental differences (scale, encoder, evaluation protocol) rather than the BM25-vs-embedding distinction.

4.3 No Downstream Signal on SWE-bench Lite

Table 3 reports end-to-end resolve rates on SWE-bench Lite (300 instances) using a Qwen3.6-27B agent. The largest gap across all conditions is 3.0pp (Qwen-embedding 25.0% vs StarCoder-dataflow 22.0%), well within the $\pm$5pp binomial confidence interval. Bootstrap resampling (10,000 iterations) confirms all pairwise comparisons are non-significant ($p > 0.80$).

With 300 binary outcomes at a base rate of $\sim$25%, the binomial standard error is $\sqrt{0.25 \times 0.75/300} \approx 0.025$, yielding a CI half-width of $\sim$5pp. Detecting a 0.5pp retrieval difference would require $N \approx 60{,}000$ instances. This is not a power problem we can fix—it is a fundamental limitation of downstream evaluation for detecting sub-1pp retrieval differences.

4.4 External Validation: RepoBench-R

Table 4 reports R@1 on RepoBench-R (easy and hard splits, $\sim$8K instances each). All confidence intervals overlap across strategies. The inconsistent sign pattern across encoders (positive for Qwen, negative for StarCoder) replicates the main result: strategy differences are noise.

4.5 Stratified Analysis

Table 5 breaks down the embedding-minus-random delta across five strata on Qwen. No stratum produces a significant effect after Bonferroni correction (10 comparisons, $\alpha_{\text{adj}} = 0.005$). The most informative finding: the 34% of instances with structural data show a tiny positive dataflow signal ($+0.40$pp), completely washed out by the 66% fallback instances.

Table 3: SWE-bench Lite resolve rates. No mining strategy produces detectable downstream improvement.

Encoder	Strategy	Resolve %	Δ vs Rand	95% Boot. CI
Qwen	Random	24.67	—	[19.9, 29.4]
	Embedding	25.00	+0.33pp	[20.2, 29.8]
	Dataflow	24.33	−0.33pp	[19.6, 29.1]
StarCoder	Random	22.67	—	[18.0, 27.3]
	Embedding	22.33	−0.34pp	[17.7, 27.0]
	Dataflow	22.00	−0.67pp	[17.3, 26.7]

Table 4: RepoBench-R R@1 (%) by strategy, encoder, and split. ± indicates 95% bootstrap CI half-width.

Split	Encoder	Random	Embedding	Dataflow	Δ E−R
Easy	Qwen	71.2±1.4	71.5±1.5	70.9±1.6	+0.3pp
	StarCoder	68.5±1.6	68.1±1.7	67.8±1.8	−0.4pp
Hard	Qwen	43.1±2.0	43.4±2.1	42.7±2.2	+0.3pp
	StarCoder	39.8±2.2	39.3±2.4	38.9±2.5	−0.5pp

4.6 Pool Size Sensitivity

Table 6 evaluates strategy rankings at pool sizes $\{50, 100, 200, 500\}$. Strategy rankings are highly unstable: the best strategy changes 3 times across 4 pool sizes (stability $= 25\%$), and Kendall's τ between pool-50 and pool-500 rankings is only 0.33. This instability holds for both encoders.

4.7 Scale-Dependent Strategy Effects

Table 8 reveals the paper's most explanatory result. At 1K training instances, embedding mining shows $+1.3$pp—consistent with pilot-scale findings. This advantage decreases monotonically to $+0.6$pp at 3K and $+0.2$pp at 5K (full scale). Dataflow mining follows the same pattern, dropping from $+0.7$pp to -0.45pp.

The mechanism is straightforward: with more training data, random negatives sample a wider region of the embedding space, effectively approximating the coverage that curated negatives provide by design. At small scale, curated negatives concentrate training signal on the decision boundary; at large scale, random sampling covers the boundary through sheer volume. This phenomenon parallels batch-size scaling effects observed in visual contrastive learning.

4.8 Failure Analysis

Manual categorization of 50 retrieval failures (cases where all strategies fail) reveals a ceiling on the potential benefit of any mining strategy (Table 9). The dominant failure mode (36%) is that the gold file is lexically dissimilar to the query, requiring fundamentally different approaches (e.g., multi-modal queries with error traces). Only 10% of failures involve cases where structural connectivity could plausibly help.

4.9 Representation Geometry Convergence

Table 10 presents the mechanistic core of our analysis. Before fine-tuning, mining strategies produce markedly different negative difficulties: on Qwen, random negatives have margin 0.074 while embedding negatives have margin 0.017 (ratio 4.35×). After fine-tuning, this ratio collapses to 1.29× (margins 0.067 vs. 0.052). Fine-tuning erases 78% of the pretrained margin difference on Qwen and 65% on StarCoder.

Table 5: Per-stratum Δ(embedding $-$ random) R@1 on Qwen. None significant after correction.

Stratum	N	**Δ R@1 (pp)**	**95% CI**
Repo small ($<$200 files)	4,596	+0.80	[$-$1.5, +3.1]
Repo large ($\geq$200 files)	39,577	$-$0.15	[$-$0.9, +0.6]
Pool small ($<$30)	12,044	+0.35	[$-$1.2, +1.9]
Pool large ($\geq$30)	32,129	$-$0.20	[$-$1.0, +0.6]
High query-pos sim ($>$0.95)	28,800	$-$0.10	[$-$0.8, +0.6]
Low query-pos sim ($\leq$0.95)	15,373	+0.55	[$-$0.9, +2.0]
Struct. available (34%)	15,133	+0.15	[$-$1.2, +1.5]
Struct. fallback (66%)	29,040	+0.25	[$-$0.5, +1.0]
Neg. diff. Q1 (easiest)	11,043	+0.60	[$-$1.5, +2.7]
Neg. diff. Q4 (hardest)	11,043	$-$0.30	[$-$2.3, +1.7]

Table 6: Strategy ranking stability across pool sizes (Qwen).

Pool	**Rand R@1**	**Emb R@1**	**DF R@1**	**Best**
50	0.529	0.531	0.525	Emb
100	0.412	0.409	0.405	Rand
200	0.324	0.327	0.325	Emb
500	0.231	0.229	0.230	Rand

The contrastive objective drives all strategies toward the same representation geometry regardless of the specific negatives used. Pretrained geometry—which determines initial negative difficulty—is a transient condition that gets erased during training. This explains why pilot-scale results showed strategy effects (insufficient training to erase geometry) while full-scale results do not (sufficient training to converge).

5 Discussion

Why strategies converge. The representation geometry analysis (§4.9) provides a unified explanation. The contrastive loss optimizes for a target geometry (margins that minimize loss), and LoRA fine-tuning has sufficient capacity to reach this target from any starting point defined by the mining strategy. Different strategies change the *path* through representation space but converge to the same *destination*. At small scale (§4.7), training is insufficient to complete this convergence, so initial differences persist and strategy choice matters. At full scale, convergence is complete and strategy choice is irrelevant.

Implications for the BM25-proxy debate. Our finding that BM25 and embedding negatives are indistinguishable (§4.2) means that prior work's reported harm cannot be attributed to the BM25-vs-embedding distinction. We observe that CoCoHaNeRe's $-$27pp effect was not reproduced in our setup with either method, suggesting other experimental factors (training scale, encoder architecture, evaluation protocol) are responsible. We frame this as an *observation* rather than a correction, as we cannot fully disentangle the confounds.

Training-time vs. inference-time structure. Our null result for dataflow-aware training-time mining contrasts with reported gains from inference-time graph navigation methods (LocAgent: $+$8–12pp on SWE-bench; CodexGraph: $+$5pp). The key difference: training-time mining fixes structural signal at corpus construction time, but the relevant structure changes per query. Inference-time methods adapt structural navigation to the specific query context. This reframes our contribution: structure matters for code retrieval, but the mechanism for incorporating it should be inference-time navigation, not training-time negative curation.

Table 7: Strategy ranking stability across pool sizes (StarCoder).

Pool	Rand R@1	Emb R@1	DF R@1	Best
50	0.443	0.440	0.428	Rand
100	0.341	0.338	0.331	Rand
200	0.262	0.264	0.259	Emb
500	0.178	0.175	0.177	Rand

Table 8: Strategy effects decrease monotonically with training scale (Qwen, 1 seed).

Train size	Rand R@1	Emb R@1	Δ (pp)	DF Δ (pp)
1K	0.505	0.518	+1.30	+0.70
3K	0.520	0.526	+0.60	+0.10
5K (full)	0.529	0.531	+0.20	-0.45

Practical recommendations.

1. If your training set exceeds $\sim$5K instances, random negative sampling is sufficient. Save the GPU time that would be spent on mining.
2. If your training set is small ($<$2K), embedding-distance mining provides a modest benefit ($+1.3$pp) worth the overhead.
3. Do not optimize mining strategy based on a single pool size—rankings are unstable (Kendall $\tau = 0.33$).
4. For leveraging code structure in retrieval, invest in inference-time graph navigation rather than training-time negative mining.
5. If structural negative mining is attempted, invest in extraction coverage first: at 34% coverage, the structural signal is too diluted to surface.

Limitations. Our study uses two 3B-parameter encoders; larger models may exhibit different convergence behavior. All training uses LoRA; full fine-tuning might preserve more strategy-specific signal. The dataflow extraction covers only 34% of instances; higher coverage might change the dataflow result, though the stratified analysis (§4.5) suggests the effect would remain small. SWE-bench Lite's 300 instances provide limited statistical power for downstream evaluation.

6 Conclusion

We presented a controlled study of hard negative mining strategies for contrastive code retrieval, finding that no strategy—embedding-distance, dataflow-aware, or BM25—reliably outperforms random baselines at full training scale. The representation geometry convergence analysis reveals the mechanism: fine-tuning acts as a geometry equalizer, collapsing pretrained margin differences by 65–78% and rendering the initial negative difficulty distribution irrelevant. Data-scaling ablations show that mining advantages observed at pilot scale vanish monotonically with training set size, consistent with scale substituting for curation.

Our negative results carry actionable implications: practitioners should prioritize training data volume over mining sophistication, use random negatives at scale, and leverage code structure through inference-time navigation rather than training-time negative selection. We hope this work encourages the community to report and value negative results in retrieval research, where publication bias toward positive findings can mislead practitioners into over-investing in techniques that do not survive rigorous evaluation.

Table 9: Failure category breakdown ($n = 50$). Structure can address only $\sim$10% of failures.

Category	Count	%
Gold file lexically dissimilar	18	36
Config / test / non-code files	12	24
Large repo dilution (>500 files)	8	16
Cross-language dependency	4	8
Structural connectivity helps	5	10
Ambiguous	3	6

Table 10: Representation geometry pre- vs. post-fine-tuning. Fine-tuning collapses margin differences across strategies.

Encoder	Phase	Rand. margin	Emb. margin	DF margin	Ratio (R/E)
Qwen	Pretrained	0.074	0.017	0.029	4.35
	Fine-tuned	0.067	0.052	0.059	1.29
StarCoder	Pretrained	0.194	0.067	0.119	2.90
	Fine-tuned	0.073	0.057	0.065	1.28

A Temperature Ablation

Table 11 shows that the null result holds across temperatures $\tau \in \{0.05, 0.10, 0.20\}$. Strategy rankings are unstable across temperatures (embedding leads at $\tau = 0.10$ but trails at $\tau = 0.20$), confirming the absence of a robust strategy effect. Higher temperatures degrade all strategies uniformly by $\sim$2.5pp at $\tau = 0.20$.

Table 11: Temperature $\times$ strategy interaction (R@1, Qwen).

τ	Random	Embedding	Δ E$-$R	Δ DF$-$R
0.05	0.529	0.531	+0.20pp	$-$0.45pp
0.10	0.522	0.525	+0.35pp	$-$0.15pp
0.20	0.504	0.503	$-$0.10pp	$-$0.25pp

B Dynamic Mining Ablation

Re-mining embedding negatives per epoch (dynamic) vs. once from the pretrained encoder (static) yields negligible improvement (Table 12): $+0.05$pp on both encoders at $3.4\times$ time overhead. The encoder changes minimally during LoRA fine-tuning (cosine similarity between epoch-0 and epoch-3 embeddings > 0.995), so dynamic and static negatives are nearly identical.

C Hop Count Ablation

Expanding call-graph scope from 1-hop to 3-hop does not rescue the dataflow strategy (Table 13). All hop counts produce R@1 within noise of random (0.529). 3-hop slightly worsens performance by over-expanding the connected set, reducing the pool of available structural negatives.

D Fusion Head Ablation

Adding structural features at inference time via a combined scoring head does not improve over embedding-only retrieval (Table 14). The structural-only head performs poorly (R@1 $= 0.312$), and the combined

Table 12: Static vs. dynamic embedding mining.

Encoder	Static R@1	Dynamic R@1	Δ (pp)	Time ×
Qwen	0.531	0.532	+0.05	3.4×
StarCoder	0.440	0.440	+0.05	3.4×

Table 13: Hop count ablation (Qwen, dataflow strategy, 1 seed).

Hops	Connected/query	Neg. available	R@1	Δ vs Rand
1	2.1	47.9	0.523	−0.60pp
2	4.8	45.2	0.525	−0.45pp
3	11.3	38.7	0.521	−0.80pp

head slightly underperforms embedding-only (-0.25pp), suggesting the model learns to partially ignore the noisy structural channel.

Table 14: Fusion head ablation (Qwen, best checkpoint).

Head	R@1	R@5	Δ vs Emb
Embedding only	0.531	0.960	—
Structural only	0.312	0.620	−21.9pp
Combined	0.529	0.955	−0.25pp

E Distance Weighting Ablation

Using dataflow-distance-weighted negative selection instead of binary connected/disconnected yields a marginal $+0.10$pp over binary, still underperforming random by 0.35pp (Table 15).

References

[1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Proc. ICLR*, 2024.

[2] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *Proc. ICLR*, 2021.

[3] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proc. EMNLP*, 2020.

[4] Tianyang Liu, Canwen Xu, and Julian McAuley. RepoBench: Benchmarking repository-level code auto-completion systems. In *Proc. ICLR*, 2024.

Table 15: Distance weighting ablation (Qwen, 1 seed).

Selection	**R@1**	**Δ vs Rand**	**Δ vs Binary**
Random	0.529	—	—
Binary disconnect	0.525	−0.45pp	—
Distance weighted	0.526	−0.35pp	+0.10pp